RetrievalRouter: Joint Modality and Architecture Selection for Document Retrieval

Emre Kuru[1], Mehmet Onur Keskin[2], Reza Farahbakhsh[1], Noel Crespi[1]
[1]SAMOVAR, Télécom SudParis, Institut Polytechnique de Paris, Palaiseau, France
[2]Özyeğin University, Istanbul, Türkiye
emre.kuru@telecom-sudparis.eu

Abstract

Document retrieval increasingly supports high-stakes information access in finance, healthcare, and law. Modern retrieval pipelines vary both in modality (text or multimodal) and in retrieval architecture (dense or late-interaction). These choices impose a hard compromise: the most effective pipelines are too slow and expensive to run at scale, while the fastest fail to retrieve evidence from complex documents. Practitioners must therefore choose between missed evidence and unusable latency, with no principled basis for adapting that choice at the query level. We show that this compromise is unnecessary. Not every query requires the same pipeline. Across benchmarks spanning financial and scientific corpora, no static pipeline dominates. We introduce RetrievalRouter, a lightweight query-aware router that learns, from the query text alone, which retrieval pipeline best fits each query. A single tunable parameter exposes the full accuracy–latency frontier, and for every static baseline, RetrievalRouter offers an operating point that is simultaneously more accurate and faster. Against the best static baseline, RetrievalRouter is 2.5% more accurate and 12.4× faster. Furthermore, compared with prior adaptive strategy selection methods, RetrievalRouter achieves significantly higher nDCG@5 across accuracy-oriented settings, while matching or numerically outperforming them on both nDCG@5 and latency in latency-oriented settings. Our code: https://github.com/emrekuruu/retrieval-router

1 Introduction

Document retrieval underpins information access in high-stakes domains such as finance (Li et al., 2025), healthcare (Xia et al., 2024), and law (Gao et al., 2024). In these settings, retrieving the right evidence is essential, as missing or irrelevant documents can lead to unsupported critical decisions.

Modern document retrieval pipelines vary along two design axes. The first is modality: text-based retrievers operate on text extracted from documents (Lin and Byrne, 2022), while multimodal retrievers (Ma et al., 2024; Faysse et al., 2024) operate directly on rendered page images. The second is architecture: dense retrievers (Karpukhin et al., 2020) compress each document into a single embedding for fast retrieval, while late-interaction architectures (Khattab and Zaharia, 2020) preserve fine-grained per-token embeddings at higher cost. These choices are not free. Accuracy and efficiency move in opposite directions across the space: configurations that handle complex documents most reliably are also the slowest, while configurations cheap enough for production traffic fail silently on those same documents (Kuru and Keskin, 2026). Without a principled way to predict which pipeline a given query needs, practitioners must commit to a single configuration at design time and pay the cost on every query.

We show that this compromise is unnecessary. Not every query requires the same pipeline. Across 11 retrieval benchmarks spanning financial, scientific, and open-domain corpora, we find that each pipeline's failures are query-dependent and asymmetric. Along the modality axis, text-based pipelines degrade sharply on queries against visually complex documents, while multimodal pipelines fall behind on queries that require nuanced textual understanding. Along the architecture axis, dense retrievers match late-interaction performance on the majority of queries at a fraction of the cost, but late-interaction architectures remain essential on the subset of queries where dense matching fails.

To address this gap, we introduce **RetrievalRouter**, a lightweight router that, given only the query text, predicts both the modality and the architecture of the retrieval pipeline to apply. To our knowledge, RetrievalRouter is the first system to route jointly across these two axes for a single underlying corpus. By directing each query to the

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 35968–35982
October 24-29, 2026 ©2026 Association for Computational Linguistics

cheapest pipeline that can answer it, the router reserves expensive configurations for queries that genuinely require them. The resulting system attains the accuracy of the strongest static pipeline while incurring an average latency near the cheapest. In summary, our main contributions are:

- A systematic empirical analysis of modern retrieval pipelines along two axes, modality and architecture, evaluated across 11 benchmarks, and characterizing the query-level strengths and failure modes of each configuration.
- RetrievalRouter, the first query-aware router that jointly selects retrieval modality and architecture per query. It dominates every static configuration on the accuracy–latency frontier, achieving 2.5% higher nDCG@5 than the strongest static baseline while being $12.4\times$ faster. Compared with prior adaptive strategy selection methods, it achieves significantly higher nDCG@5 in accuracy-oriented settings, while numerically outperforming them in both effectiveness and latency in latency-oriented settings.
- A query-level pipeline-selection benchmark, releasing per-query best-pipeline labels across more than 80,000 queries to support further research on adaptive retrieval.

2 Related Work

This work builds on two lines of prior research: the static retrieval pipelines whose design space we route over, and the adaptive retrieval strategies that explore query-time decision-making.

Early document retrieval systems applied dense retrievers to extracted document text (Lin and Byrne, 2022; Karpukhin et al., 2020). To recover the document structure lost in extraction, subsequent work introduced layout-aware chunking that segmented documents into semantic regions such as headers and tables (Yepes et al., 2024). The text-extraction step itself remains a fundamental ceiling: Zhang et al. (2025) shows that extraction errors propagate through the pipeline causing systematic failures on layout-sensitive queries that dense text embeddings cannot recover.

Multimodal retrievers emerged as a response, encoding rendered page images directly and avoiding text extraction. MuRAG (Chen et al., 2022) introduced joint image-text retrieval through a multimodal memory; later work embedded both modalities into a unified representation space and enabled dense retrieval over raw visual inputs (Ma et al., 2024; Riedler and Langer, 2024). ColPali (Faysse et al., 2024) extended the ColBERT late-interaction paradigm to visual patches, achieving state-of-the-art accuracy on visually complex benchmarks. However, visual encoding has its own ceiling: VTCBench (Zhao et al., 2025) shows that vision-language models degrade substantially on tasks requiring long-range textual reasoning, falling behind text-only retrievers in such settings.

A growing body of work relaxes the assumption that a fixed retrieval pipeline serves every query. Adaptive approaches differ in *what they adapt*. One line decides *whether* to retrieve or *how deeply*, conditioning on query difficulty, uncertainty, or learned policies (Asai et al., 2024; Tang et al., 2025), while keeping the retriever itself fixed. A second line routes across heterogeneous knowledge bases or distinct source modalities (Peng et al., 2025; Jiang et al., 2025), where each corpus serves a different semantic role.

Closer to our setting, a recent line of work routes across different retrievers over a single corpus. Arabzadeh et al. (2021) train a query-only classifier to choose between sparse and dense retrieval using hard strategy labels derived from retrieval success. LiteGator (Darmanto and Atimi, 2025) switches between sparse and dense retrieval per query under a latency budget. RouterRetriever (Lee et al., 2025) routes among domain-specific LoRA experts within a single dense architecture. MoR (Kalra et al., 2025) ensembles sparse and dense retrievers with per-query trust weights computed from pre- and post-retrieval signals.

3 Background

We evaluate seven retrieval pipelines spanning two modalities (text and multimodal) and four retrieval strategies (sparse, dense, late-interaction, and reranking). The five core pipelines define the design space; the two reranking variants represent the systems deployed in practice.

Text pipelines. Sparse pipelines (BM25) apply term-based matching to extracted page text (Robertson et al., 2009). Unlike neural retrievers, BM25 requires no embedding generation, making it the lowest-latency pipeline in our design space. Dense pipelines (TD) embed the same extracted page text into a single vector with a bi-encoder (Linq-Embed-Mistral (Kim et al., 2024)).

Their learned representations capture semantic similarity beyond exact term overlap, improving retrieval when queries and relevant pages use different wording, but embedding generation makes TD slower than BM25. Late-interaction pipelines (TL) swap the bi-encoder for a late-interaction model (GTE-ModernColBERT (Chaffin, 2025)) and preserve per-token embeddings, enabling finer-grained matching at higher cost. TL is the most accurate core text pipeline, but also the slowest. All three text pipelines inherit the limitations of text extraction: extraction errors and layout loss cap their accuracy on visually structured documents (Zhang et al., 2025). Because none of the text pipelines can perceive non-textual content, we caption charts, figures, and tables with a vision-language model (Gemini 3.0 Flash (Team et al., 2023)) and concatenate the descriptions with the extracted text before indexing (Zhao et al., 2023), so any remaining performance gap reflects architectural limits rather than missing content.

Multimodal pipelines. Multimodal-Dense (MD) operates directly on full-page images without text extraction, encoding each page into a single visual embedding (Nomic-Embed-Multimodal (Team, 2025a)). It is the fastest neural pipeline overall and the natural multimodal counterpart to TD. Multimodal-Late (ML) keeps the visual input but represents each page as a bag of patch embeddings (ColNomic-Embed-Multimodal (Team, 2025a)) and applies patch-level late interaction at retrieval time. It is the most accurate pipeline, but also the slowest.

Reranking pipelines. Reranking is the standard industry technique for deploying late-interaction at scale, recovering nearly all of its accuracy at a fraction of its latency (Nogueira and Cho, 2019): a fast dense retriever fetches a small candidate set (k=100), and the late-interaction model is applied only to those candidates. We include both rerank variants because they reflect how late-interaction is actually deployed in practice and they constitute the strongest deployable static baselines our router must surpass. Text-Rerank (TR) combines Linq-Embed-Mistral with GTE-ModernColBERT, and Multimodal-Rerank (MR) combines Nomic-Embed-Multimodal with ColNomic-Embed-Multimodal.

4 Proposed Approach

The seven retrieval pipelines evaluated in Section 3 occupy different points on the accuracy–latency frontier, and no single pipeline dominates. Any fixed choice, therefore, forces one trade-off on every query: overspend on queries it can handle easily, or underperform on queries it handles poorly. We propose RetrievalRouter, a lightweight query-aware policy that resolves this by selecting a retrieval pipeline for each query. From the query text alone, it predicts which pipeline best fits the query.

Figure 1 illustrates this on three representative examples. The first, a visual reference query, asks about a chart identified by its color, a property that text-only retrieval cannot perceive, so multimodal capabilities are required. The second, a textual factoid, can be resolved through direct term matching, so BM25 avoids unnecessary embedding generation. The third spans a long document with evidence at both ends; dense retrieval averages the document into a single vector, losing this distant structure, while late-interaction preserves the token-level vectors that capture evidence across the document. Each decision picks the cheapest pipeline that can answer the query, and the savings from easy queries make expensive configurations affordable for hard ones. No fixed-pipeline system has this property.

We use a single router over the five routing arms rather than splitting the decision into a modality router and an architecture router. Pipelines are not fully described by their modality and architecture alone; each has its own strengths and trade-offs, and the signals about which one fits a query live in the query's latent representation. A router with direct access to all five arms can learn these patterns; one that operates at a higher level does not see the full picture at once.

4.1 Problem Formulation

Let $\mathcal{P} = \{\text{BM25}, \text{TD}, \text{TL}, \text{TR}, \text{MD}, \text{ML}, \text{MR}\}$ denote the seven evaluated pipelines. In preliminary analyses, we found that the reranking pipelines matched or nearly matched the effectiveness of their pure late-interaction counterparts at substantially lower latency. Including both variants would introduce near-redundant actions and make the routing signal harder to learn. We therefore define the router action space as $\mathcal{A} = \{\text{BM25}, \text{TD}, \text{TR}, \text{MD}, \text{MR}\} \subset \mathcal{P}$, while retaining TL and ML as static baselines.

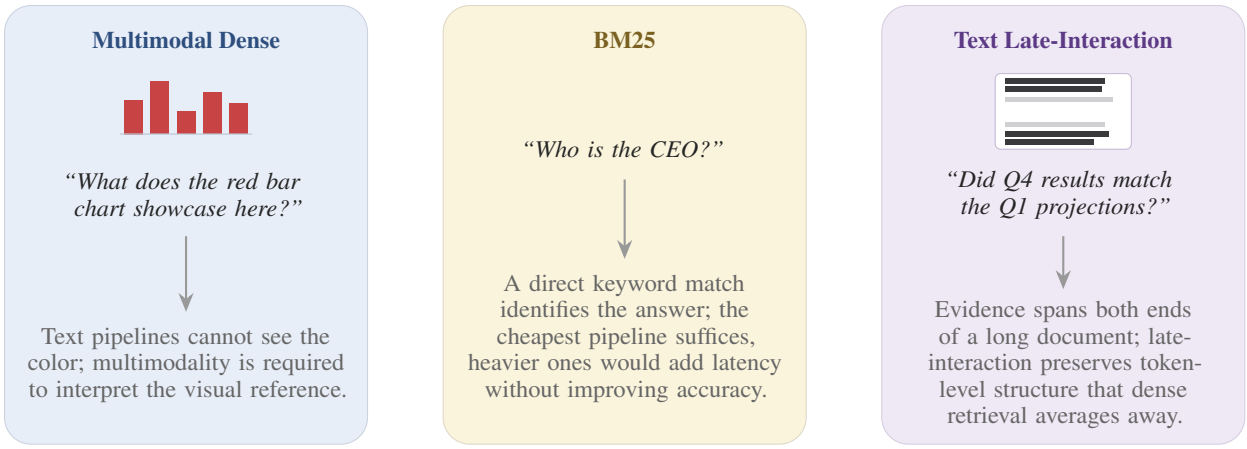


Figure 1: Qualitative routing examples. The router sends queries to the cheapest pipeline capable of answering it.

The router is a parameterized policy $\pi_\theta(\cdot \mid q)$ that maps a query q to a distribution over $\mathcal{A}$. At inference time, we route q to the pipeline $\arg\max_{p_i \in \mathcal{A}} \pi_\theta(p_i \mid q)$. For each routing arm $p_i \in \mathcal{A}$ and query q, we define a per-query reward that combines accuracy and efficiency:

$$r_i(q) = (1 - \lambda) \cdot s_i(q) + \lambda \cdot \big(1 - \ell_i(q)\big) \quad (1)$$

where $s_i(q) \in [0, 1]$ is the nDCG@5 of routing arm p_i on query q and $\ell_i(q) \in [0, 1]$ is its per-query normalized latency, $\ell_i(q) = \frac{L_i(q)}{\sum_{p_j \in \mathcal{A}} L_j(q)}$. Here, $L_i(q)$ is the raw wall-clock latency of routing arm p_i, and the denominator is the total latency across the five routing arms in $\mathcal{A}$. The efficiency score is then represented by $1 - \ell_i(q)$. The hyperparameter $\lambda \in [0, 1]$ controls the accuracy–latency trade-off: $\lambda = 0$ yields a quality-only objective, and $\lambda \to 1$ favors the fastest routing arm. Sweeping λ at training time traces out the accuracy–latency Pareto frontier.

4.2 Router Architecture

The router has two components: a query encoder E_ϕ and a lightweight decision head H_ψ. We use Qwen3-0.6B-Base (Team, 2025b) as the encoder, with LoRA adapters (Hu et al., 2022) on the attention and feedforward projections; the base weights remain frozen during training. We mean-pool the encoder's final hidden states into a 1024-dimensional query representation z_q. The decision head is a single linear layer that maps z_q to logits $\mathbf{s} \in \mathbb{R}^{|\mathcal{A}|}$ over the five routing arms, and the output distribution is a softmax over the logits, $\pi_\theta(p_i \mid q) = \exp(s_i) / \sum_{p_j \in \mathcal{A}} \exp(s_j)$.

4.3 Training

Training on hard labels (the single best pipeline per query) is unreliable for this problem. On any given query, multiple pipelines often retrieve the same documents and tie on nDCG@5, and forcing the router to pick one arbitrary winner injects label noise that the model has no way to resolve. We train instead against a soft target derived from the full per-query reward vector over $\mathcal{A}$.

Oracle labels. For each training query q, we construct the reward vector over the five routing arms, $\mathbf{r}_\mathcal{A}(q) = [r_i(q)]_{p_i \in \mathcal{A}}$, using Equation 1.

Soft targets. The reward vector $\mathbf{r}_\mathcal{A}(q)$ is converted to a target probability distribution over the routing arms via softmax:

$$\tilde{p}_i(q) = \frac{\exp(r_i(q)/\tau)}{\sum_{p_j \in \mathcal{A}} \exp(r_j(q)/\tau)} \quad (2)$$

where $\tau > 0$ is a temperature scaling parameter. We set $\tau = 0.1$. Because the rewards $r_i(q)$ are bounded in $[0, 1]$ and concentrate in a narrow band when most pipelines succeed, a standard softmax ($\tau = 1$) yields near-uniform targets and dilutes the training signal, especially on easy queries where the only differentiator is latency. A small τ amplifies these gaps into a clear preference while exact ties remain exact ties. Queries with all-zero rewards are excluded from the gradient. At $\lambda = 0$, this removes queries for which every arm fails; at $\lambda > 0$, the latency term provides an efficiency signal, so these queries remain in training.

Objective. The router minimizes the KL divergence between its predicted policy and the target:

$$\mathcal{L}(\theta) = \sum_{q \in \mathcal{D}_{\text{train}}} D_{\text{KL}}\big(\tilde{\mathbf{p}}(q) \,\big\|\, \pi_\theta(\cdot \mid q)\big). \quad (3)$$

The router thus learns to approximate the per-query oracle decision at inference, without running any of the pipelines themselves. Implementation details are provided in Appendix A.

5 Experimental Setup

We evaluate RetrievalRouter across 11 benchmarks spanning financial, scientific, and open-domain corpora. All retrieval pipelines, router training, and inference are run on the same NVIDIA H100 80GB GPU, ensuring latency measurements are directly comparable across systems.

5.1 Datasets

We use 11 datasets spanning text-heavy scientific papers, mixed-modality financial reports, and chart-dense slide decks, drawn from three benchmarks. Dataset statistics are reported in Appendix A.

- **REAL-MM-RAG** (Wasserman et al., 2025): *FinReport* and *FinSlides*. Real-world financial documents.
- **T2-RAGBench** (Strich et al., 2025): *FinQA*, *ConvFinQA*, *VQAonBD*, and *TAT-DQA*. Hybrid reasoning over text and tables.
- **MMDocRAG** (Dong et al., 2025): *ArxivQA*, *Wiki-SS*, *MP-DocVQA*, *SciQAG*, and *DUDE*. Long-context textual retrieval.

5.2 Baselines

We compare RetrievalRouter against the seven static pipelines from Section 3 (BM25, TD, TL, TR, MD, ML, MR), each of which is run identically on every query. These establish the accuracy–latency frontier achievable without query-aware routing.

We also compare against the closest existing adaptive baseline to our setting, the query-level retrieval strategy-selection method introduced by Arabzadeh et al. (2021). The original method trains a hard-label classifier to select a sparse retriever when it ranks a relevant document above a fixed threshold and otherwise escalate to a dense retriever. We extend this rule to our five-pipeline set by ordering the pipelines by their measured mean latency and assigning each training query to the cheapest pipeline that ranks a relevant page in the first position. Queries for which no pipeline succeeds are assigned to BM25. At inference time, a probability threshold controls whether a query remains on BM25 or is escalated to the classifier's highest-scoring neural pipeline. Sweeping this threshold produces the baseline's accuracy–latency frontier. Here, budget denotes the fraction of queries routed away from BM25 to a neural pipeline. For comparison with RetrievalRouter, we report the baseline on the equivalent scale $\lambda = 1 - \text{budget}$, so lower λ corresponds to a larger escalation budget.

Finally, we report a seven-pipeline per-query Oracle as an upper bound. For each test query, the Oracle selects from the full evaluated set $\mathcal{P}$, including TL and ML, the pipeline that maximizes the reward from Equation 1. For this upper bound, latency is normalized across all pipelines in $\mathcal{P}$ rather than the five routing arms in $\mathcal{A}$.

5.3 Metrics

Effectiveness. We report nDCG@5, MRR@5, and Recall@5.

Latency. End-to-end wall-clock time per query, including router inference, embedding generation, and vector search. We report both mean and P95 latencies to capture tail behavior.

Statistical testing. We assess normality of paired-difference distributions using the D'Agostino–Pearson test, applying a paired t-test when normal and a two-sided Wilcoxon signed-rank test otherwise. We control family-wise error across planned comparisons using Holm correction. All statistical significance is reported at $p < 0.001$.

Storage. We report the disk footprint of the four neural vector indices maintained by RetrievalRouter. Rerank pipelines reuse the corresponding dense and late-interaction indices and therefore require no additional vector storage. BM25 uses a separate lexical index, which is not included in the vector-store comparison.

6 Evaluation

Table 1 reports retrieval effectiveness and end-to-end latency for the seven static pipelines. Table 2 compares RetrievalRouter, Arabzadeh et al. (2021), and the Oracle at six values of λ. Figure 2 plots the static pipelines and deployable adaptive methods in the accuracy–latency plane.

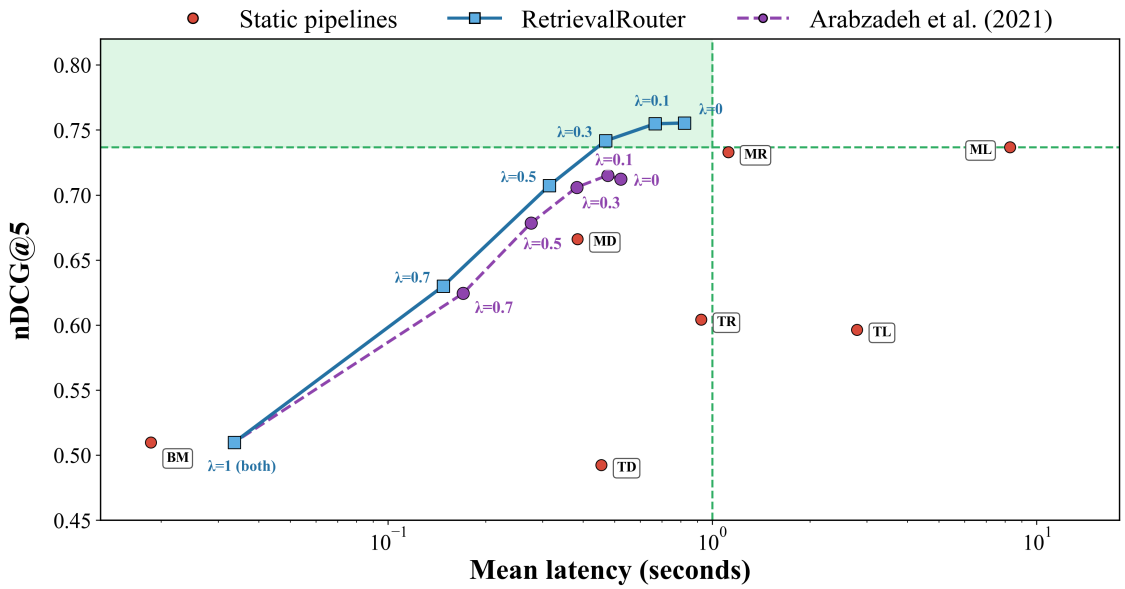


Figure 2: Accuracy–efficiency operating points for all methods. The shaded area marks the wanted region: higher accuracy than the most accurate static pipeline with mean latency below 1s.

Static pipelines. ML is the most effective static pipeline, reaching 0.737 nDCG@5, but is also the slowest at 8.283s per query. MR nearly preserves this effectiveness (0.733 nDCG) while reducing mean latency to 1.121s. MD provides a lower-cost middle ground at 0.666 nDCG and 0.385s. At the opposite extreme, BM25 is by far the fastest pipeline at 0.019s, but reaches only 0.510 nDCG. No static pipeline therefore combines the effectiveness of late interaction with the latency of lightweight retrieval.

RetrievalRouter dominates the neural static baselines on both axes. Sweeping λ along the accuracy–latency frontier yields operating points that dominate every neural static pipeline. At $\lambda = 0.1$, RetrievalRouter reaches 0.755 nDCG@5 at 0.666s and dominates all four late-interaction pipelines. Relative to the multimodal variants, it improves effectiveness by 2.5% over ML and 3.0% over MR while being $12.4\times$ and $1.7\times$ faster, respectively. Relative to the text variants, it improves effectiveness by 26.5% over TL and 24.9% over TR while being $4.2\times$ and $1.4\times$ faster. At $\lambda = 0.5$, RetrievalRouter reaches 0.707 nDCG@5 at 0.314s and dominates both dense pipelines, improving effectiveness by 6.2% over MD and 43.6% over TD while being $1.2\times$ and $1.4\times$ faster. Finally, at $\lambda = 1$, RetrievalRouter selects BM25 for every query and reaches 0.034s total latency after policy inference. All gains are significant ($p < 0.001$).

Router overhead. Routing incurs 15 ms of overhead. This cost becomes meaningful in total latency only when a substantial share of queries is routed to BM25. At $\lambda = 1$, where every query is routed to BM25, the overhead increases the latency from 0.019s to 0.034s; even then, it remains faster than every neural static pipeline.

RetrievalRouter outperforms prior adaptive routing. In the accuracy-oriented settings ($\lambda = 0$ through 0.5), RetrievalRouter achieves significantly higher nDCG@5 than the strategy-selection baseline of Arabzadeh et al. (2021), while the baseline remains significantly faster (all $p < 0.001$). As λ increases, RetrievalRouter's effectiveness gain narrows from 0.043 to 0.029 nDCG and its latency overhead falls from 0.299s to 0.038s. At the latency-oriented setting $\lambda = 0.7$, RetrievalRouter numerically improves both effectiveness (0.630 vs. 0.624 nDCG) and mean latency (0.148s vs. 0.171s), although neither difference is significant (nDCG: $p = 0.021$; latency: $p = 0.015$). At $\lambda = 1$, both methods select BM25 for every query and therefore converge to the same endpoint.

Oracle headroom. The per-query oracle reaches 0.90 nDCG@5, leaving roughly 14 nDCG points of additional headroom available with perfect routing decisions. Although the oracle is not a deployable system, this gap demonstrates how far a query-aware router could plausibly go, given the current set of static pipelines.

Model	nDCG	MRR	Recall	Latency	P95
BM25	0.510	0.476	0.613	**0.019**	**0.046**
TD	0.492	0.456	0.601	0.455	0.845
TR	0.604	0.572	0.702	0.924	1.665
TL	0.597	0.560	0.706	2.796	5.717
MD	0.666	0.629	0.779	0.385	0.792
MR	0.733	0.701	0.830	1.121	1.967
ML	**0.737**	**0.704**	**0.834**	8.283	17.744

Table 1: Aggregate performance of the static retrieval pipelines. Bold indicates the best static pipeline.

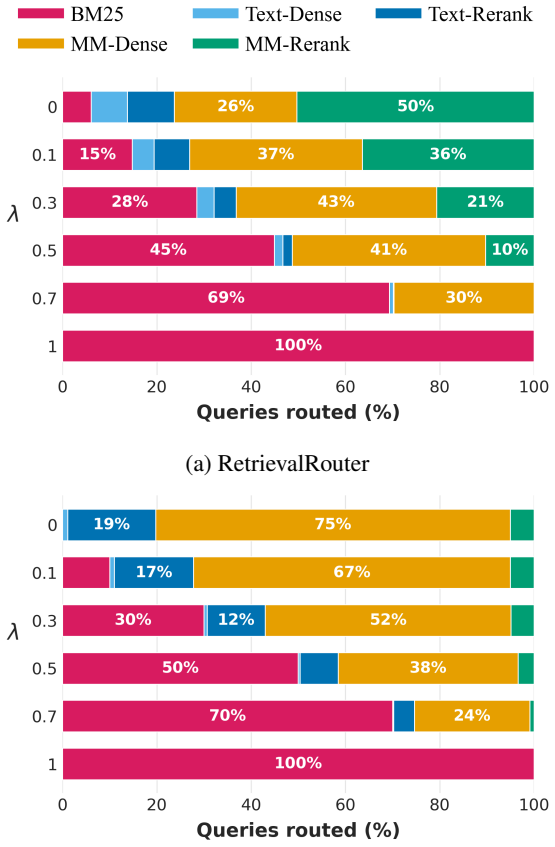
	RetrievalRouter		Arabzadeh et al.		Oracle	
λ	nDCG	Latency	nDCG	Latency	nDCG	Latency
0.00	**0.755**†	0.821	0.712	**0.522**†	0.901	0.927
0.10	**0.755**†	0.666	0.715	**0.476**†	0.901	0.340
0.30	**0.742**†	0.469	0.706	**0.382**†	0.901	0.335
0.50	**0.707**†	0.314	0.678	**0.276**†	0.893	0.306
0.70	**0.630**	**0.148**	0.624	0.171	0.829	0.197
1.00	0.510	0.034	0.510	0.034	0.510	0.019

Table 2: Aggregate performance of the adaptive retrieval pipelines. Bold indicates the better deployable adaptive method. † denotes significance ($p < 0.001$).

6.1 Routing Distribution

Pipeline allocation shifts across the frontier. To understand how the adaptive strategy-selection baseline of Arabzadeh et al. (2021) and RetrievalRouter produce their respective accuracy–efficiency frontiers, we analyze how each method distributes queries across the five selectable retrieval pipelines. Figure 3 compares these routing distributions as the preference shifts from accuracy to efficiency.

Soft targets enable flexible routing. The difference in routing behavior follows directly from the two training objectives. The strategy-selection baseline assigns each query a single hard label corresponding to the cheapest pipeline that successfully retrieves a relevant page. It therefore cannot become entirely accuracy focused, because its labels reward the first pipeline that succeeds rather than the pipeline that achieves the highest retrieval quality. Even at the equivalent $\lambda = 0$ setting, where all emphasis is placed on accuracy, the baseline assigns most queries to MD, the cheapest neural pipeline that succeeds for many queries. This leaves performance on the table when a heavier reranking pipeline could produce a better ranking. RetrievalRouter instead learns from soft targets derived from the complete per-query reward vector. At $\lambda = 0$, the latency term vanishes and the targets reflect only retrieval quality, allowing the router to select expensive reranking pipelines whenever their accuracy gains justify them. As λ increases, the targets gradually shift toward MD and BM25, enabling RetrievalRouter to adapt its allocation to the desired accuracy–efficiency trade-off.



(a) RetrievalRouter

(b) Arabzadeh et al. (2021)

Figure 3: Pipeline-selection distributions of adaptive methods across the accuracy–efficiency frontier.

Cheaper pipelines are not always inferior. Even at the fully accuracy-focused setting of $\lambda = 0$, RetrievalRouter assigns 6.0% of queries to BM25. Because latency has no influence in this setting, these selections show that the router learns where sparse term matching is effective rather than treating BM25 merely as a low-cost fallback. In contrast, the strategy-selection baseline uses its budget to control how many queries remain on BM25, treating the sparse pipeline as replaceable as more computational budget becomes available. At its equivalent $\lambda = 0$ setting, every query is therefore routed to a neural pipeline, preventing BM25 from being selected even when it is the best fit.

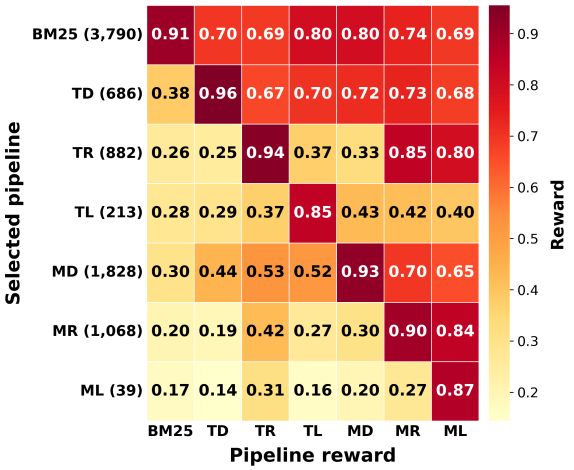
The per-dataset results reinforce this distinction; even at $\lambda = 0$, RetrievalRouter routes 20.0% of

Wiki-SS queries to BM25, where BM25 outperforms four of the six neural static pipelines, while the adaptive baseline routes none to BM25 (Appendix B, Figure 7).

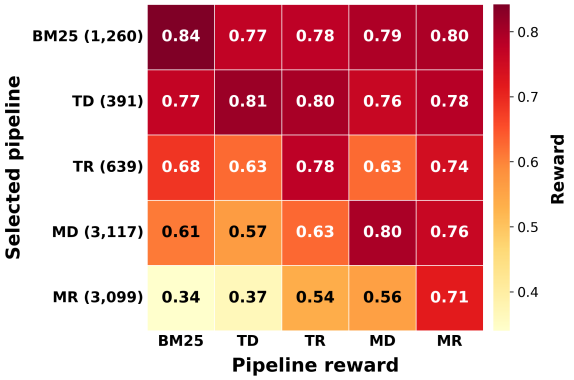
6.2 Router Decision Quality

To verify that the router learns meaningful selection, we visualize routing decisions as reward heatmaps in Figure 4. Each row corresponds to a selected pipeline, and each column reports the reward it received on the same set of queries. We use $\lambda = 0.1$, since it is closest to the Pareto knee.

Diagonal dominance. Both the oracle and the router produce strong diagonal patterns: when a pipeline is selected, it is close to optimal for the queries it received. Non-selected pipelines consistently underperform on the same queries. The router does not pick certain pipelines globally; it activates each pipeline selectively for queries where it is the right choice.



(a) Oracle ($\lambda = 0.1$)



(b) RetrievalRouter ($\lambda = 0.1$)

Figure 4: Reward heatmaps for Oracle and RetrievalRouter decisions. Rows indicate the selected pipeline and columns report the reward of each pipeline.

Pure late-interaction is rarely chosen. The Oracle selects TL for 213 queries and ML for 39, compared with 882 for TR and 1,068 for MR. The rerank variants therefore capture most queries where fine-grained interaction is useful, although pure late interaction remains optimal for a very small subset.

Reranking is not always necessary. Not every query benefits from reranking either; in some cases, adding the rerank pass adds unnecessary cost or even actively degrades retrieval quality. The clearest example is Multimodal-Dense. Its aggregate accuracy across the suite (0.67 nDCG@5) is much lower than its late-interaction counterparts (0.74 nDCG@5), yet on the 1,828 queries where the Oracle selects it, MD achieves an average reward of 0.93. Adding the rerank step (MR) on those same queries drops the reward to 0.70.

6.3 Modality Sensitivity

To understand why no single modality suffices across our benchmark suite, we examine how text and multimodal pipelines respond to visual complexity and textual nuance. To characterize the visual complexity of each dataset, we compute a per-page visual density score using DocLayout-YOLO (Zhao et al., 2024), defined as the fraction of page area covered by non-textual elements (tables, figures, charts). Scores lie in $[0, 1]$, with higher values indicating heavier visual content.

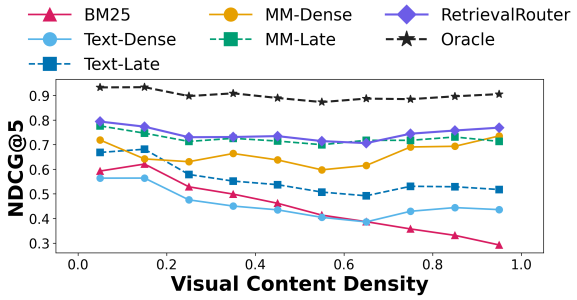
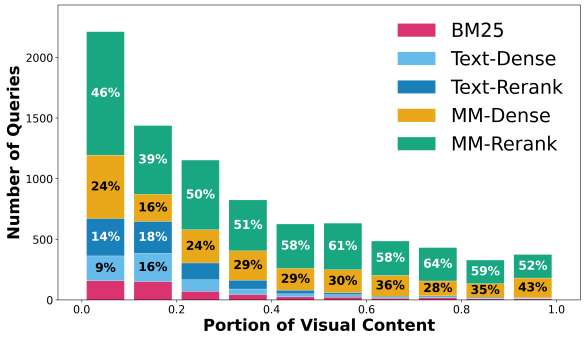


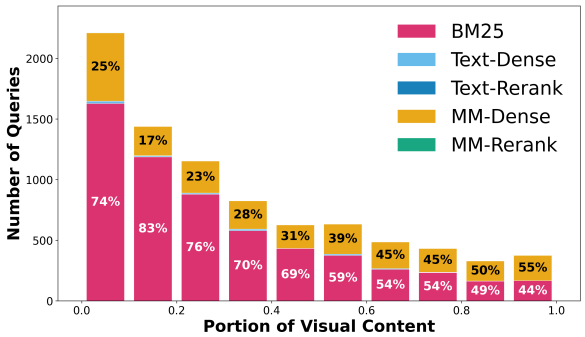
Figure 5: nDCG@5 across visual content density.

Text pipelines degrade under visual complexity. Figure 5 shows that BM25 is the most sensitive to visual content, falling from 0.593 nDCG@5 in the lowest-density bin to 0.292 in the highest. TD and TL also decline as density rises because text extraction linearizes documents and discards the spatial structure needed for tables, charts, and figures. In contrast, the multimodal pipelines remain substantially more stable.

RetrievalRouter tracks this gradient (Figure 6): at $\lambda = 0$, the share of queries routed to multimodal pipelines rises from 69.7% to 95.2% across the density range. Even at $\lambda = 0.7$, BM25 usage falls from 73.6% to 44.3%, while MD rises from 25.4% to 55.5%. Visual complexity influences routing even when the objective prioritizes efficiency.



(a) $\lambda = 0$



(b) $\lambda = 0.7$

Figure 6: RetrievalRouter pipeline-selection distributions across visual-density bins.

Multimodal pipelines struggle with textual nuance. The reverse failure also exists. Table 5 shows that multimodal pipelines do not dominate every dataset. On Wiki-SS, TL reaches 0.784 nDCG@5, outperforming both ML (0.743) and MR (0.732). BM25 also reaches 0.745, exceeding both multimodal late-interaction variants. Wiki-SS emphasizes long-range textual reasoning and fine-grained semantic integration, a regime in which visual encoders' patch-level structure can become a liability. RetrievalRouter recognizes this structure and reaches 0.785 nDCG@5, slightly exceeding the best static pipeline. Multimodal representations are essential for layout-heavy documents, but can dilute the sequential textual structure that language-intensive tasks depend on.

7 Conclusions

In this work, we showed that the accuracy–latency trade-off in document retrieval is a routing problem, not an architectural one. RetrievalRouter formalizes this view: it dispatches each query to the cheapest pipeline that can answer it, reserving expensive pipelines for queries that genuinely require them. The practical consequence is that practitioners no longer face a binary choice between an accurate pipeline that is too slow to deploy and a fast pipeline that fails on the documents in their workload.

Our contributions are threefold: (i) a systematic empirical analysis of modern retrieval pipelines along the modality and architecture axes across 11 benchmarks; (ii) RetrievalRouter itself, the first system to perform joint modality–architecture routing over a single underlying corpus; and (iii) a query-level pipeline-selection benchmark releasing per-query best-pipeline labels across more than 80,000 queries.

Our analysis of static pipelines surfaces several findings. (i) Late-interaction pipelines outperform their dense counterparts across the suite, but (ii) their rerank variants recover all of that accuracy essentially at a fraction of the cost, making reranking the best choice for deployment of late-interaction. (iii) Reranking is nonetheless not always beneficial: it adds unnecessary latency when the dense retriever already answers the query correctly, and in some cases actively degrades performance. On the modality axis, (iv) text-based pipelines degrade under high visual complexity, while (v) multimodal pipelines degrade under textual nuance. Finally, (vi) lexical retrieval remains competitive on datasets where exact keyword matching is effective, making BM25 a viable accuracy-oriented choice rather than merely a low-cost fallback.

RetrievalRouter recognizes and avoids these failure modes. It dominates every neural static configuration across both accuracy and latency: 2.5% higher nDCG@5 than the strongest static baseline while being 12.4× faster, and 3.0% higher than the deployment-standard rerank variant at 1.7× lower latency. Furthermore, compared with prior adaptive strategy selection methods, RetrievalRouter achieves significantly higher nDCG@5 in accuracy-oriented settings, while matching or numerically outperforming them in both effectiveness and latency in latency-oriented settings.

Limitations

The Storage Cost. A primary criticism of multi-model routing involves operational storage costs. Unlike a static pipeline that maintains a single index, RetrievalRouter maintains four vector indices and a separate BM25 lexical index. Table 3 reports the vector index sizes (excluding lexical BM25). The storage footprint is dominated by the multimodal-late (ColPali) index, which holds multi-vector patch embeddings. For the combined corpus, the multimodal-late index ($\approx$ 39 GB) is more than $13\times$ larger than the text-dense index ($\approx$ 3 GB). This trade-off is fundamental: our approach uses more storage for reduced inference latency. In cloud environments where storage is cheap and latency directly affects user experience and API costs, caching representations minimizes demands on costly GPU compute time. In contrast, storage-constrained environments may benefit from a static pipeline. Against this backdrop, it is essential to examine alternative approaches, such as cascading systems, and compare them with routing.

Pipeline	Index Size (MB)	Relative Factor
Text-Dense	3,003	$1.0\times$
Text-Late	8,308	$2.8\times$
Multimodal-Dense	2,618	$0.9\times$
Multimodal-Late	39,023	$13.0\times$

Table 3: Vector-index size comparison.

The VRAM Cost. Similarly, another key trade-off in our approach is the increased GPU usage required to achieve lower latency. Maintaining all pipelines at once requires substantially more VRAM (40 GB) than standard approaches. Consequently, our approach is best suited for *latency-critical* applications where hardware costs are secondary to user experience.

Cross-Domain Generalization. Our experimental setup uses an intra-dataset split (80/10/10), evaluating in-domain generalization where query distributions are known at deployment time. A potential limitation is that the router may learn domain-specific lexical cues (for instance, financial vocabulary as a proxy for high visual density) rather than the underlying semantic structure. In practice, enterprise document search engines operate over fixed corpora with known query domains at compile time, making in-domain splits representative of production scenarios. Nonetheless, evaluating zero-shot cross-domain generalization, such as testing on multimodal visual document benchmarks like ViDoRe (Loison et al., 2026), remains a valuable direction for future research.

Query-Only Semantic Ambiguity. A key limitation of RetrievalRouter is its reliance on the query text alone. While the router outperforms all static baselines, a substantial gap remains relative to a relevance-aware Oracle because the query text alone can be semantically ambiguous. For example, queries like "summarize the table on page 5" are linguistically identical regardless of whether the referenced document contains a visual infographic or extracted text. Purely semantic routing is thus underdetermined when the optimal pipeline depends on the target document's latent layout rather than query intent. Overcoming this ceiling requires active probing, such as exploratory dense retrieval or partial metadata inspection, to resolve document-level structural cues before full pipeline dispatch. Finally, our routing action space focuses on bi-encoder and late-interaction architectures; expanding the policy to decide when expensive generative LLM-based rerankers (e.g., listwise or pointwise reasoning) or downstream generator model sizes are justified represents a compelling direction for compound AI systems.

Ethical Considerations

We foresee no major ethical concerns or potential risks in our work. All retrieval models, encoders, and benchmark datasets used in this study are publicly released and open-sourced, and are used under their original licenses. We introduce no new human-subject data and conduct no human evaluation.

Data and Reproducibility. The 11 benchmarks span financial filings, scientific papers, and open-domain web sources. We release our code, training scripts, and per-query oracle labels across more than 80,000 queries to support reproducibility and further work on adaptive retrieval. Our code and data are available at `https://github.com/emrekuruu/retrieval-router`

Bias and Generalization. The training distribution is restricted to English-language documents from financial, scientific, and open-domain corpora. Practitioners deploying the router outside these distributions should evaluate performance on representative in-domain queries.

References

Negar Arabzadeh, Xinyi Yan, and Charles LA Clarke. 2021. Predicting efficiency/effectiveness trade-offs for dense vs. sparse retrieval strategy selection. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, pages 2862–2866.

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-rag: Learning to retrieve, generate, and critique through self-reflection.

Antoine Chaffin. 2025. Gte-moderncolbert.

Wenhu Chen, Hexiang Hu, Xi Chen, Pat Verga, and William W Cohen. 2022. Murag: Multimodal retrieval-augmented generator for open question answering over images and text. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5558–5570, Abu Dhabi, UAE. Association for Computational Linguistics.

Ar-Razy Muhammad Darmanto and Rizqia Lestika Atimi. 2025. Litegator: A lightweight and budget-aware retriever routing framework for low-resource question answering. In *International Conference on Applied Science and Technology on Engineering Science 2025 (iCAST-ES 2025)*, pages 687–695. Atlantis Press.

Kuicai Dong, Yujing Chang, Xin Deik Goh, Dexun Li, Ruiming Tang, and Yong Liu. 2025. Mmdocir: Benchmarking multi-modal retrieval for long documents. *Preprint*, arXiv:2501.08828.

Manuel Faysse, Hugues Sibille, Tony Wu, Bilel Omrani, Gautier Viaud, Céline Hudelot, and Pierre Colombo. 2024. Colpali: Efficient document retrieval with vision language models. *Preprint*, arXiv:2407.01449.

Cheng Gao, Chaojun Xiao, Zhenghao Liu, Huimin Chen, Zhiyuan Liu, and Maosong Sun. 2024. Enhancing legal case retrieval via scaling high-quality synthetic query-candidate pairs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 7086–7100.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, and 1 others. 2022. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3.

Zhuohang Jiang, Pangjing Wu, Xu Yuan, Wenqi Fan, and Qing Li. 2025. Qa-dragon: Query-aware dynamic rag system for knowledge-intensive visual question answering. In *Proceedings of the KDD 2025 Cup Workshop*, pages 1–12, Toronto, ON, Canada. Association for Computing Machinery.

Jushaan Singh Kalra, Xinran Zhao, To Eun Kim, Fengyu Cai, Fernando Diaz, and Tongshuang Wu. 2025. Mor: Better handling diverse queries with a mixture of sparse, dense, and human retrievers. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 11982–12001.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, Online. Association for Computational Linguistics.

Omar Khattab and Matei Zaharia. 2020. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48, New York, NY, USA. Association for Computing Machinery.

Junseong Kim, Seolhwa Lee, Jihoon Kwon, Sangmo Gu, Yejin Kim, Minkyung Cho, Jy-yong Sohn, and Chanyeol Choi. 2024. Linq-embed-mistral: Elevating text retrieval with improved gpt data through task-specific control and quality refinement. Linq AI Research Blog.

Emre Kuru and Mehmet Onur Keskin. 2026. Evaluating modern rag: Textual, multimodal, dense, and late interaction pipelines. *Preprint*, arXiv:2608.23176.

Hyunji Lee, Luca Soldaini, Arman Cohan, Minjoon Seo, and Kyle Lo. 2025. Routerretriever: Routing over a mixture of expert embedding models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 11995–12003.

Ying Li, Mengyu Wang, Miguel de Carvalho, Sotirios Sabanis, and Tiejun Ma. 2025. Fingear: Financial mapping-guided enhanced answer retrieval. *Preprint*.

Weizhe Lin and Bill Byrne. 2022. Retrieval augmented visual question answering with outside knowledge. *Preprint*, arXiv:2210.03809.

António Loison, Quentin Macé, Antoine Edy, Victor Xing, Tom Balough, Gabriel de Souza P. Moreira, Bo Liu, Manuel Faysse, Celine Hudelot, and Gautier Viaud. 2026. ViDoRe v3: A comprehensive evaluation of retrieval augmented generation in complex real-world scenarios. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16570–16600, San Diego, California, United States. Association for Computational Linguistics.

Xueguang Ma, Sheng-Chieh Lin, Minghan Li, Wenhu Chen, and Jimmy Lin. 2024. Unifying multimodal retrieval via document screenshot embedding. *Preprint*, arXiv:2406.11251.

Rodrigo Nogueira and Kyunghyun Cho. 2019. Passage re-ranking with bert. *arXiv preprint arXiv:1901.04085*.

Chunyi Peng, Zhipeng Xu, Zhenghao Liu, Yishan Li, Yukun Yan, Shuo Wang, Zhiyuan Liu, Yu Gu, Minghe Yu, and Ge Yu. 2025. Learning to route queries across knowledge bases for

step-wise retrieval-augmented reasoning. *Preprint*, arXiv:2505.22095.

Monica Riedler and Stefan Langer. 2024. Beyond text: Optimizing rag with multimodal inputs for industrial applications. *Preprint*, arXiv:2410.21943.

Stephen Robertson, Hugo Zaragoza, and 1 others. 2009. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389.

Jan Strich, Enes Kutay Isgorur, Maximilian Trescher, Chris Biemann, and Martin Semmann. 2025. T^2-ragbench: Text-and-table benchmark for evaluating retrieval-augmented generation. *Preprint*, arXiv:2506.12071.

Xiaqiang Tang, Qiang Gao, Jian Li, Nan Du, Qi Li, and Sihong Xie. 2025. Mba-rag: A bandit approach for adaptive retrieval-augmented generation through question complexity. In *Proceedings of the 31st International Conference on Computational Linguistics (COLING)*, pages 3248–3254, Abu Dhabi, UAE. Association for Computational Linguistics.

Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, and 1 others. 2023. Gemini: A family of highly capable multimodal models. *Preprint*, arXiv:2312.11805.

Nomic Team. 2025a. Nomic embed multimodal: Interleaved text, image, and screenshots for visual document retrieval.

Qwen Team. 2025b. Qwen3 technical report. *Preprint*, arXiv:2505.09388.

Navve Wasserman, Roi Pony, Oshri Naparstek, Adi Raz Goldfarb, Eli Schwartz, Udi Barzelay, and Leonid Karlinsky. 2025. Real-mm-rag: A real-world multimodal retrieval benchmark. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 31660–31683, Vienna, Austria. Association for Computational Linguistics.

Peng Xia, Kangyu Zhu, Haoran Li, Hongtu Zhu, Yun Li, Gang Li, Linjun Zhang, and Huaxiu Yao. 2024. Rule: Reliable multimodal rag for factuality in medical vision language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 1081–1093.

Antonio Jimeno Yepes, Yao You, Jan Milczek, Sebastian Laverde, and Renyu Li. 2024. Financial report chunking for effective retrieval augmented generation. *Preprint*, arXiv:2402.05131.

Junyuan Zhang, Qintong Zhang, Bin Wang, Linke Ouyang, Zichen Wen, Ying Li, Ka-Ho Chow, Conghui He, and Wentao Zhang. 2025. Ocr hinders rag: Evaluating the cascading impact of ocr on retrieval-augmented generation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 1–12, San Diego, CA, USA. IEEE.

Hongbo Zhao, Meng Wang, Fei Zhu, Wenzhuo Liu, Bolin Ni, Fanhu Zeng, Gaofeng Meng, and Zhaoxiang Zhang. 2025. Vtcbench: Can vision-language models understand long context with vision-text compression? *Preprint*, arXiv:2512.15649.

Ruochen Zhao, Hailin Chen, Weishi Wang, Fangkai Jiao, Xuan Long Do, and 1 others. 2023. Retrieving multimodal information for augmented generation: A survey. *Preprint*, arXiv:2303.10868.

Zhiyuan Zhao, Hengrui Kang, Bin Wang, and Conghui He. 2024. Doclayout-yolo: Enhancing document layout analysis through diverse synthetic data and global-to-local adaptive perception. *arXiv preprint arXiv:2410.12628*.

A Experimental Details

Dataset Statistics

Table 4 reports the query count, corpus size, and average document length for each dataset.

Benchmark	Subset	# Queries	# Docs	Avg. Tokens
REAL-MM-RAG	FinReport	853	2,687	1,053
	FinSlides	1,048	2,280	275
T2-RAGBench	FinQA	6,232	2,789	965
	ConvFinQA	3,431	1,806	966
	VQAnBD	9,772	1,787	780
	TAT-DQA	27,127	2,758	852
MMDocRAG	ArxivQA	9,034	4,749	1,110
	Wiki-SS	14,968	12,752	777
	MP-DocVQA	5,581	2,350	388
	SciQAG	4,496	2,595	1,196
	DUDE	2,561	2,073	516

Table 4: Dataset statistics. The suite covers over 80,000 queries across financial, scientific, and open domains.

Implementation Details

Data split. Each source dataset is independently split into 80% train, 10% validation, and 10% test. The test set is held out during oracle labeling and router training.

Oracle generation. For each training query, all seven pipelines are executed once to record nDCG@5 and latency. While the base nDCG and latency metrics are collected once via the oracle, the composite reward and soft targets are dynamically recomputed for each specific λ objective during training using Equations 1 and 2.

Hyperparameters. The decision head is a single linear projection from the 1024-dimensional query representation to the five routing logits, with dropout 0.1 applied to the pooled representation. The LoRA adapters use rank 16, $\alpha = 32$, and dropout 0.05, introducing approximately 4M trainable parameters out of Qwen3-0.6B's 600M total. Queries are tokenized to a maximum of 128 tokens. We train for 2 epochs using AdamW with learning rate 1×10^{-4}, batch size 16 with gradient accumulation of 2 (effective batch size 32), weight decay 0.01, and linear warmup over the first 10% of steps followed by cosine decay, on a single NVIDIA H100 in bfloat16 precision. Training takes approximately 20 minutes per λ setting. LoRA and optimizer hyperparameters follow standard practice for LoRA fine-tuning; the temperature $\tau = 0.1$ was selected on the validation split. Finally, all models are trained with a random seed of 42, and the reported test-set numbers are from a single training run per λ setting.

Inference Cost. On an NVIDIA H100, the router adds approximately 15ms of inference latency per query. For perspective, the Text-Dense pipeline runs in roughly 400ms end-to-end, while Multimodal-Late runs in roughly 8s. The routing overhead therefore corresponds to approximately 3.8% of TD latency and under 0.2% of ML latency.

B Per-Dataset Analysis

Table 5 reports per-dataset nDCG@5, MRR@5, and Recall@5 for the static pipelines, adaptive methods, and Oracle. Figure 7 compares the pipeline allocations of RetrievalRouter and Arabzadeh et al. at $\lambda = 0$, showing how the two training objectives respond to dataset-specific retrieval demands.

From our results, we observe that RetrievalRouter allocates computation based on where each pipeline improves retrieval rather than following a fixed cost ordering. Arabzadeh et al. routes 84–100% of queries from the six financial datasets to MD because its hard-label objective favors the cheapest pipeline that succeeds. However, MR is the strongest static pipeline on all six datasets. RetrievalRouter recognizes this difference and routes 60–90% of their queries to MR instead (Table 5a). The allocation changes when a dataset favors another modality. On Wiki-SS, where text pipelines outperform multimodal pipelines, RetrievalRouter sends 94% of queries to the text pipelines.

The results also show that RetrievalRouter learns when cheaper pipelines are appropriate. Lexical matching is particularly effective on Wiki-SS, where BM25 outperforms four of the six neural static pipelines. RetrievalRouter routes 20.0% of Wiki-SS queries to BM25 even at $\lambda = 0$, when latency has no influence on the reward. This shows that BM25 is selected for its retrieval effectiveness rather than merely as a low-cost fallback. On MP-DocVQA and DUDE, RetrievalRouter instead divides most queries between MD and MR, reflecting query-level variation within the same dataset. This dataset-aware allocation translates into broader gains: RetrievalRouter exceeds the strongest static pipeline on five datasets for nDCG@5 and MRR@5 and on four for Recall@5, while Arabzadeh et al. does so on only two datasets for each metric.

	Static Pipelines							Adaptive Routers		Upper Bound
Dataset	**BM25**	**TD**	**TL**	**TR**	**MD**	**ML**	**MR**	**Arab.**	**Ours**	**Oracle**
FinReport	0.456	0.527	0.726	0.736	0.621	0.803	**0.807**	0.620	0.798	0.938
FinSlides	0.183	0.410	0.387	0.382	0.799	0.902	**0.903**	0.799	0.886	0.949
FinQA	0.685	0.707	0.746	0.785	0.831	0.865	**0.868**	0.831	0.865	0.961
ConvFinQA	0.663	0.716	0.782	0.808	0.815	0.862	**0.863**	0.815	0.863	0.950
VQAonBD	0.595	0.564	0.626	0.731	0.815	0.846	**0.847**	0.816	0.856	0.948
TAT-DQA	0.361	0.331	0.446	0.428	0.610	0.652	**0.656**	0.633	0.678	0.842
ArxivQA	0.307	0.251	0.399	0.393	0.424	**0.664**	0.630	0.566	0.634	0.801
Wiki-SS	0.745	0.702	**0.784**	0.774	0.648	0.743	0.732	0.761	0.785	0.968
MP-DocVQA	0.441	0.423	0.604	0.628	**0.728**	0.724	0.726	0.740	0.781	0.906
SciQAG	0.722	0.731	0.871	0.879	0.804	0.881	**0.881**	0.815	0.859	0.994
DUDE	0.400	0.490	0.606	0.547	0.628	0.644	**0.647**	0.717	0.730	0.901

(a) nDCG@5

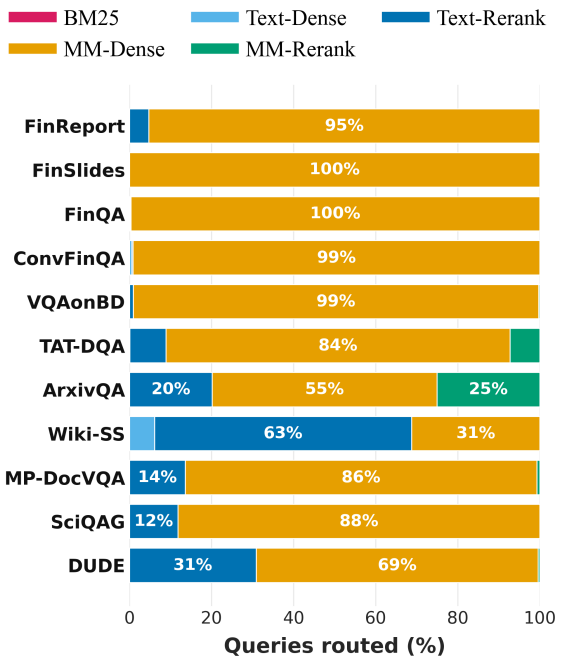
	Static Pipelines							Adaptive Routers		Upper Bound
Dataset	**BM25**	**TD**	**TL**	**TR**	**MD**	**ML**	**MR**	**Arab.**	**Ours**	**Oracle**
FinReport	0.420	0.484	0.682	0.690	0.574	0.764	**0.770**	0.576	0.758	0.921
FinSlides	0.170	0.374	0.365	0.358	0.763	0.876	**0.876**	0.763	0.857	0.932
FinQA	0.644	0.652	0.702	0.744	0.792	0.833	**0.835**	0.792	0.833	0.948
ConvFinQA	0.618	0.669	0.744	0.774	0.781	0.835	**0.836**	0.781	0.836	0.938
VQAonBD	0.549	0.514	0.575	0.686	0.774	0.810	**0.812**	0.775	0.822	0.936
TAT-DQA	0.328	0.299	0.408	0.394	0.567	0.613	**0.617**	0.591	0.637	0.812
ArxivQA	0.286	0.227	0.373	0.370	0.393	**0.639**	0.608	0.534	0.606	0.779
Wiki-SS	0.710	0.672	**0.751**	0.743	0.617	0.712	0.701	0.732	0.757	0.959
MP-DocVQA	0.413	0.388	0.573	0.606	0.697	0.699	**0.700**	0.707	0.751	0.899
SciQAG	0.679	0.685	0.841	0.850	0.766	0.855	**0.855**	0.776	0.827	0.992
DUDE	0.379	0.465	0.586	0.529	0.600	0.614	**0.617**	0.695	0.700	0.889

(b) MRR@5

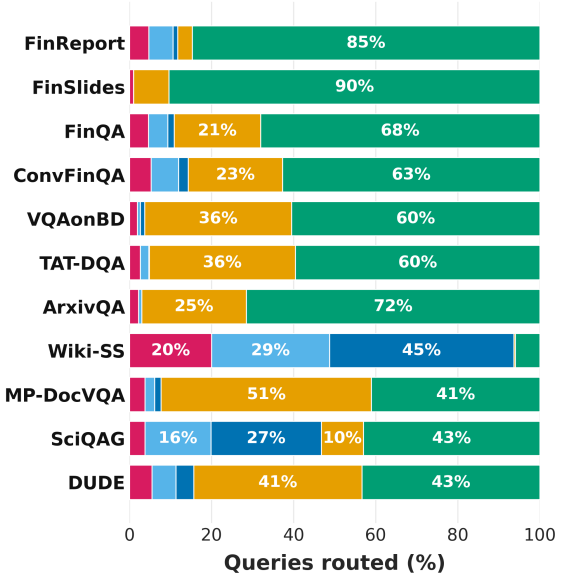
	Static Pipelines							Adaptive Routers		Upper Bound
Dataset	**BM25**	**TD**	**TL**	**TR**	**MD**	**ML**	**MR**	**Arab.**	**Ours**	**Oracle**
FinReport	0.565	0.659	0.859	0.871	0.765	**0.918**	**0.918**	0.753	0.918	0.988
FinSlides	0.221	0.519	0.452	0.452	0.904	**0.981**	**0.981**	0.904	0.971	1.000
FinQA	0.806	0.870	0.878	0.909	0.945	0.960	**0.965**	0.945	0.961	0.997
ConvFinQA	0.796	0.857	0.895	0.907	0.913	0.942	**0.945**	0.913	0.942	0.985
VQAonBD	0.735	0.715	0.778	0.865	0.934	**0.950**	**0.950**	0.936	0.956	0.985
TAT-DQA	0.461	0.427	0.561	0.529	0.740	0.767	**0.773**	0.762	0.803	0.934
ArxivQA	0.371	0.326	0.476	0.461	0.515	**0.735**	0.694	0.660	0.717	0.868
Wiki-SS	0.848	0.792	**0.885**	0.864	0.743	0.836	0.822	0.848	0.867	0.992
MP-DocVQA	0.525	0.537	0.704	0.705	**0.840**	0.809	0.813	0.847	0.876	0.947
SciQAG	0.849	0.866	0.962	**0.964**	0.918	0.958	0.958	0.931	0.955	1.000
DUDE	0.466	0.569	0.664	0.607	0.718	0.736	**0.740**	0.792	0.823	0.939

(c) Recall@5

Table 5: Per-dataset effectiveness at $\lambda = 0$. Bold marks the best static pipeline; underline marks an adaptive router that strictly outperforms the best static pipeline. Oracle values are excluded from highlighting.



(a) Arabzadeh et al. (2021)



(b) RetrievalRouter

Figure 7: Per-dataset routing distributions at $\lambda = 0$.